Alternatively, you can use the `@inject` directive to inject the `IAntiforgery` service into the page and use its `GetAndStoreTokens` method to generate a token set and access the resulting `RequestToken` property.

Listing 11.13 Generating a request verification token from the `IAntiforgery` service

```
@using Microsoft.AspNetCore.Antiforgery
@inject IAntiforgery antiforgery
@{
    var token = antiforgery.GetAndStoreTokens(HttpContext).RequestToken;
}
```

This approach is only suitable if your JavaScript code is embedded in the Razor page because you need to render the C# token variable within the script. You cannot include C# code in external JavaScript files, so if you prefer to keep the script out of your pages, you will have to use the `AntiForgeryToken` method to render the hidden field.

Next include the token within the request, either as a form field (posting form values) or as a request header value. The default name of the form field is `__RequestVerificationToken` (with two leading underscores), and the default name of the request header is `RequestVerificationToken` (with no leading underscores). Redo this example without the form element to illustrate. First, remove the form tags from the property details partially, so the inputs are no longer enclosed within a form. Next, add a call to the `Html.AntiForgeryToken` method to the City Razor page. I placed mine just above the partial tag helper:

```
@Html.AntiForgeryToken()
<partial name="_PropertyModalPartial" />
@section scripts{
```